

東京電機大学

未来科学部 ロボット・メカトロニクス学科

メカトロニクス設計製作

テーマ名「車車間通信による交差点の合流」

2026 年

目次

1. 目的
 2. 概要
 3. 基礎知識
 - 3.1 ESP-NOW
 - 3.2 V2I/V2V 通信
 4. 実験内容
 - 4.1 無線通信実験
 - 4.1.1 手順
 - 4.2 車両の走行実験
 - 4.2.1 手順
 5. 考察
 - 5.1 考察課題 1
 - 5.2 考察課題 2
- 付録
- A. test プログラム
 - B. 走行実験用プログラム

1. 目的

本実験では通信可能な 2 台の車両を用いて交差点での合流を想定した衝突回避実験を行ない、無線通信についての理解を深めることを目的とする。

2. 概要

本実験では、無線アドホック通信を用いて、合流する 2 つの車両の衝突回避実験を通して、無線通信の知識の理解を深める。はじめに、無線通信とマルチプロセスに関するプログラムの作成を通して、これらの基礎知識を学習する。次に、実機である車両を走行させるプログラムを作成し、その動作を確認する。最後に、走行距離や通信範囲などの条件を変えてプログラムを実行し、実機の動作を確認する。

3. 基礎知識

3.1 ESP-NOW

ESP-NOW は、Espressif 社が開発した無線通信プロトコルであり、Wi-Fi の物理層および MAC 層を利用して、アクセスポイントやルータを介さずに端末間で直接通信を行うことができる。ESP32 シリーズをはじめとする Espressif 社製 SoC で利用可能であり、低遅延かつ省電力なデバイス間通信を実現できる。

ESP-NOW は Wi-Fi ドライバ上で動作する独自プロトコルであるため、利用するには ESP-NOW の初期化に先立ち、Wi-Fi を Station (STA) モードに設定する必要がある。通信相手は MAC アドレスによって識別され、送信先デバイスはあらかじめ Peer として登録しておく必要がある。

1 回の送信で扱えるデータサイズは最大 250 B であり、送信するデータはバイト列として渡す必要がある。実際の送受信処理は Wi-Fi ドライバによって非同期に実行されるため、送信完了時および受信時には、それぞれ送信コールバック関数と受信コールバック関数が呼び出され、通信結果や受信データを処理する仕組みとなっている。

3.2 V2I/V2V 通信

近年、高速道路へ合流する際に多発する衝突事故の防止策として、自動運転技術を用いて衝突を回避する試みに関しての研究がさかんに行なわれている。その中で通信速度が担保されている手法として V2I(Vehicle to Infrastructure)通信が挙げられる。図 1 に V2I 通信の全体図を示す。V2I 通信は、車を検知したセンサが基地局を介して加速車線の車両へデータを送信し、加速車線の車両が速度を制御する手法である。高速道路を走行中の車がセンサの前を通過したとき、センサは基地局へ向けて高速道路を走行中の車のデータを送信する。基地局は合流したい車に向けて受け取ったデータを送る。これにより、自動運転

での合流が可能になる。しかし、V2I 通信を実現するには、合流地点にセンサや基地局を設置しなければならない、コストがかかる。本実験では、車両が互いに自身のもつ情報を直接通信する V2V(Vehicle to Vehicle)通信により、基地局や外部センサを用いずに自動運転による合流を実現する。

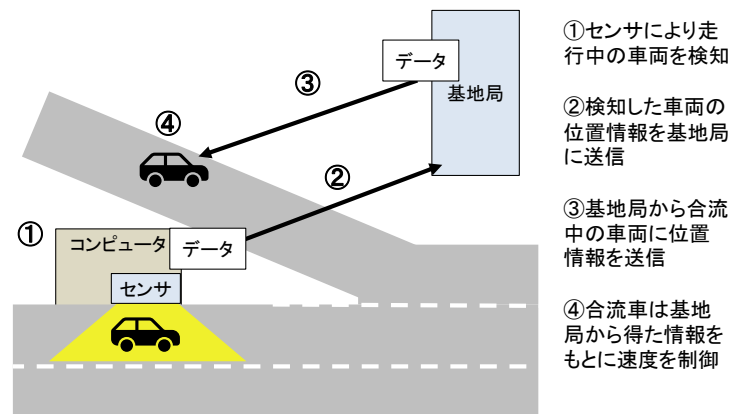


図 1 V2I 通信

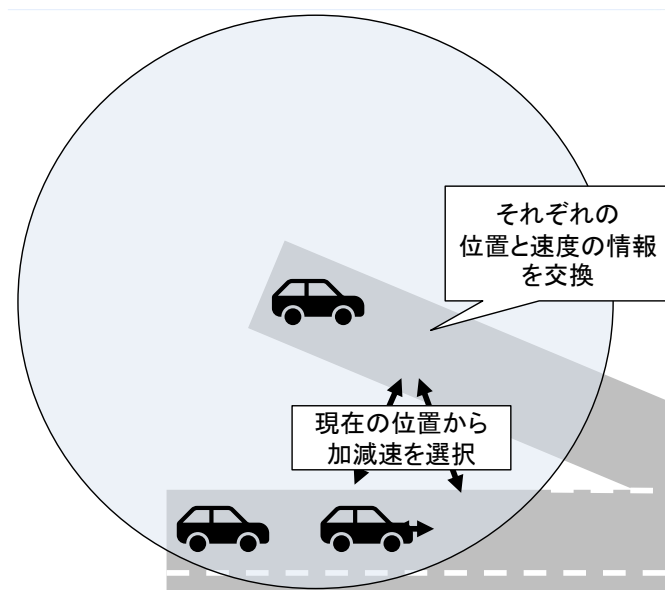


図 2 V2V 通信

4. 実験内容

本実験では、車両が保有する各種情報を無線通信により交換し、衝突せずに合流を行なうためのシステムを構築する。図5に本実験で用いる実験車両を示す。本実験では、Seed XIAO ESP32C3を搭載した2台の車両の現在の速度と合流地点までの距離情報を交換する。その情報をもとにそれぞれが衝突判定を行ない、速度を制御し合流を行なう。

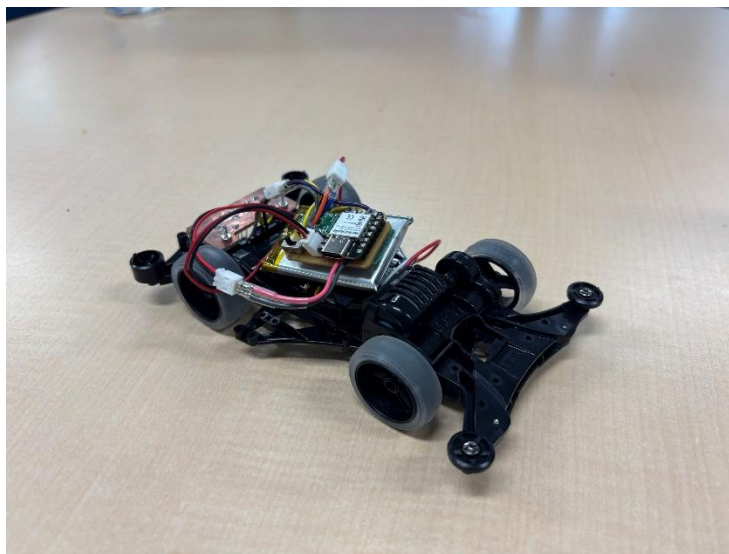


図3 実験車両

第1回ではESP-NOWの導通実験を行う。第2回では車両制御を行うためのコントローラを設計する。第3回では、2つの車両が情報を交換し、その情報に基づいて速度制御を行ない合流するプログラムを作成する。このとき、複数の条件下でその動作を確認する。

4.1 実行環境構築

本節では、あらかじめ配布されたプロジェクトを展開して実行環境の構築を行う。

4.1.1 手順

- (1) 配布された zip ファイルを展開する。
- (2) Visual Studio Code を起動する。
- (3) 拡張機能の Python, Pylance, Python Debugger, Python Enviroments, PlatfromIO IDE, C/C++, C/C++ Extension Pack をインストールする。
- (4) VSCode で展開したプロジェクトフォルダを開く。
- (5) ターミナルから.\setup.ps1 を実行する。

4.2 ESP-NOW の通信実験

2 台の Seeed XIAO ESP32C3 を使用して ESP-NOW による通信実験を行う。

4.2.1 手順

- (1) 2 台の PC と 2 台の ESP32C3 を用意する。
- (2) 図 4 に示す場所をクリックして, "env:espnow"を選択する。

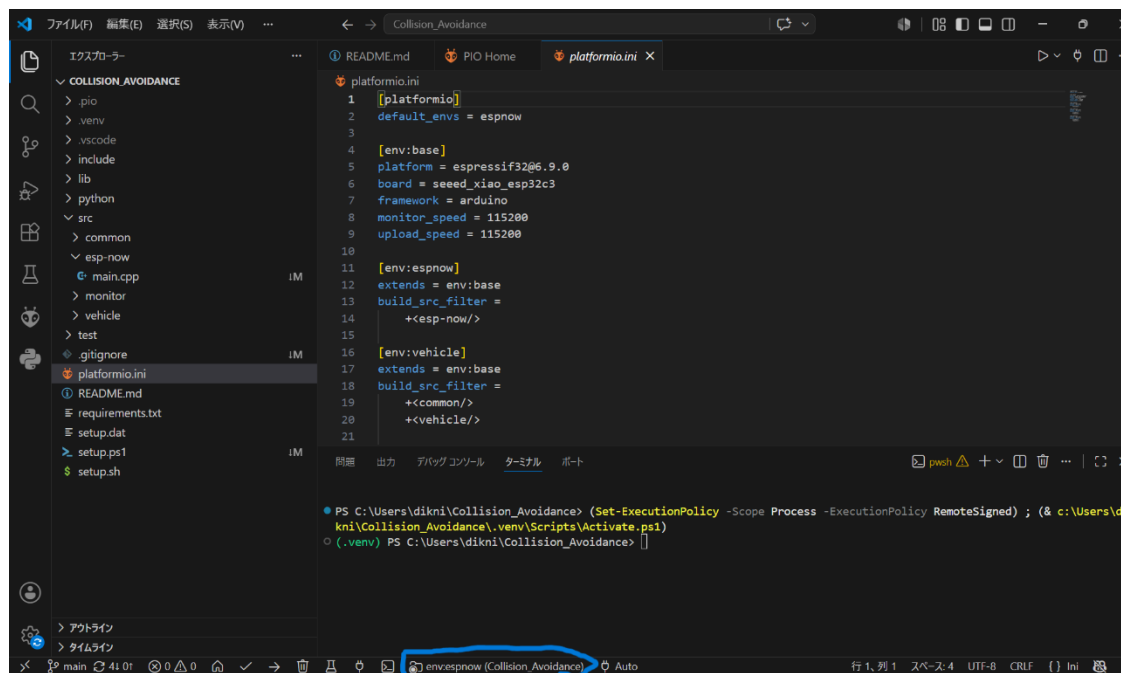


図 4 環境設定ボタンの場所

- (3) src/espnow/main.cpp の穴埋めを行う。
- (4) PC と ESP32C3 をケーブルで接続する。
- (5) 図 5 で示す場所をクリックして upload を実行する。このとき, エラーが発生する場合は, ケーブルを一度抜いて boot ボタンを押し込んだままケーブルを接続すると解決する場合がある。この場合は, upload 後に一度ケーブルを抜き挿しする。

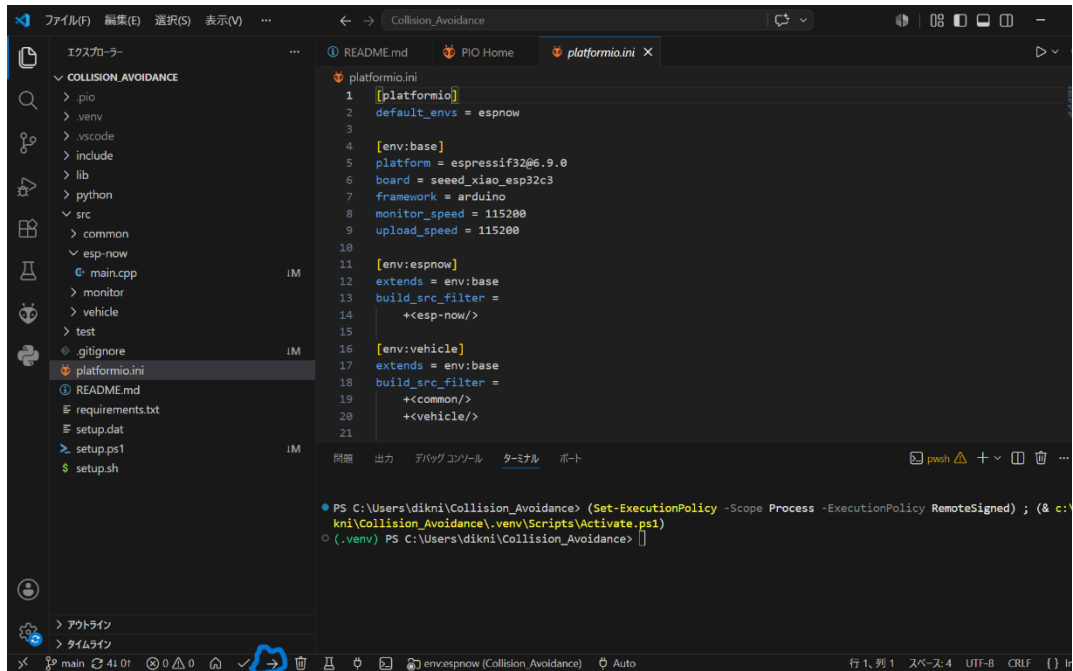


図 5 upload ボタンの場所

- (6) python/exp1_monitor.py を実行する。
- (7) COMPORT で/dev/ttyACM*を選択して Connect を押してシリアル通信を開始する。
- (8) HELLO を送信し合って相手の MAC アドレスを確認する。
- (9) 相手の MAC アドレスに PING を送信して導通を確認する。
- (10) 相手の MAC アドレスを AddPeer する。
- (11) 相手の MAC アドレスに PING を送信して導通を確認する。
- (12) 相手の MAC アドレスに DATA REQUEST を送信してダミーデータの受信を確認する。
- (13) 計測が終わったら Disconnect でシリアル通信を終了する。

4.3 車両の走行実験

実験車両を走行させて、目標速度に追従するコントローラを設計する実験を行う。

4.3.1 手順

- (1) モニタ用の ESP32C3 と実験車両を用意する。
- (2) src/vehicle/controller.cpp の PIDcontroller 関数を設計する。
- (3) PlatformIO の環境を env:vehicle に設定する。
- (4) PC と実験車両に搭載された ESP32C3 をケーブルで接続する。

- (5) upload を実行する。このとき、エラーが発生する場合は、ケーブルを一度抜いて boot ボタンを押し込んだままケーブルを接続すると解決する場合がある。この場合は、upload 後に一度ケーブルを抜き挿しする。
- (6) 実験車両のバッテリーを接続して、電源をつけてからコースに設置する。
- (7) PlatformIO の環境を `env:monitor` に設定する。
- (8) PC とモニタ用 ESP32C3 をケーブルで接続する。
- (9) upload を実行する。このとき、エラーが発生する場合は、ケーブルを一度抜いて boot ボタンを押し込んだままケーブルを接続すると解決する場合がある。この場合は、upload 後に一度ケーブルを抜き挿しする。
- (10) `python/exp2_monitor.py` を実行する。
- (11) COMPORT で `/dev/ttyACM*` を選択して **Connect** を押してシリアル通信を開始する。
- (12) `targetspeed` を設定して **SETSPEED** を送信する。
- (13) `KP`, `KI`, `KD` を設定して **SETGAIN** を送信する。
- (14) **START** を送信して計測を開始する。
- (15) **FINISH** を送信して計測を終了する。
- (16) log ディレクトリに生成されたログデータを確認する。
- (17) (12)から(16)までを繰り返してコントローラのゲインを設計する。
- (18) 計測が終わったら **Disconnect** でシリアル通信を終了する。

4.4 衝突回避実験

2 台の実験車両を同時に走行させて交差点に突入させる実験を行う。

4.4.1 手順

- (1) モニタ用の ESP32C3 と 2 台の実験車両を用意する。
- (2) 実験 3 で設計した PID ゲインを元に `include/config.hpp` にある `F_KP`, `F_KI`, `F_KD` を編集する。
- (3) `src/vehicle/computavoidance.cpp` の `ComputAvoidanceSpeed` 関数を設計する。
- (4) PlatformIO の環境を `env:vehicle` に設定する。
- (5) PC と実験車両に搭載された ESP32C3 をケーブルで接続する。
- (6) upload を実行する。このとき、エラーが発生する場合は、ケーブルを一度抜いて boot ボタンを押し込んだままケーブルを接続すると解決する場合がある。この場合は、upload 後に一度ケーブルを抜き挿しする。
- (7) 実験車両のバッテリーを接続して、電源をつけてからコースに設置する。
- (8) PlatformIO の環境を `env:monitor` に設定する。
- (9) PC とモニタ用 ESP32C3 をケーブルで接続する。

- (10) upload を実行する。このとき、エラーが発生する場合は、ケーブルを一度抜いて boot ボタンを押し込んだままケーブルを接続すると解決する場合がある。この場合は、upload 後に一度ケーブルを抜き挿しする。
- (11) python/exp3_monitor.py を実行する。
- (12) COMPORT で/dev/ttyACM*を選択して Connect を押してシリアル通信を開始する。
- (13) CollisionFlag を ON にして START を送信して、動作を確認する。
- (14) FINISH を送信して計測を終了する。
- (15) log ディレクトリに生成されたログデータを確認する。
- (16) CollisionFlag を OFF にして START を送信して、動作を確認する。
- (17) FINISH を送信して計測を終了する。
- (18) log ディレクトリに生成されたログデータを確認する。
- (19) 計測が終わったら Disconnect でシリアル通信を終了する。

5. 考察

5.1 考察課題 1

ComputAvoidanceSpeed 関数で受信した相手車両の交差点突入時間と交差点離脱時間からどのような判断で衝突回避を行っているのか、自身の実装をもとに述べよ。

5.2 考察課題 2

なぜ ESP-NOW では IP アドレスではなく MAC アドレスを用いて相手を識別しているのか考察せよ。また、IP アドレスを用いた通信と比較して ESP-NOW の利点と欠点を述べよ。

5.3 考察課題 3

ESP-NOW の通信遅延やパケットロスが発生した場合、相手の位置情報が実際の状態より古くなる。これによって衝突回避判断にどのようなリスクが生じるか考察せよ。

付録

A. 衝突回避プロジェクトの設定

衝突回避実験で用いるプロジェクトの設定ファイルを以下に示す.

表 1 衝突回避プロジェクトの設定ファイル一覧

プログラム名	内容
platformio.ini	PlatformIO IDE の設定
requirements.txt	Python プロジェクトに必要なライブラリー一覧
setup.ps1	windows で環境設定を行うプログラム
setup.sh	Linux で環境設定を行うプログラム

以下に各設定ファイルをリスト 1, リスト 2, リスト 3, テスト 7, テスト 8, テスト 9 に示す.

リスト platformio.ini

```
[platformio]
default_envs = espnow

[env:base]
platform = espressif32@6.9.0
board = seeed_xiao_esp32c3
framework = arduino
monitor_speed = 115200
upload_speed = 115200

[env:espnow]
extends = env:base
build_src_filter =
    +<esp-now/>

[env:vehicle]
extends = env:base
build_src_filter =
    +<common/>
    +<vehicle/>

[env:monitor]
extends = env:base
build_src_filter =
    +<common/>
    +<monitor/>
```

リスト 2 requirements.txt

```
pyserial>=3.5
```

リスト 3 setup.ps1

```
$ErrorActionPreference = "Stop"

Write-Host "=====
Write-Host "    Project Setup"
Write-Host "=====

#-----
# Python Virtual Environment
#-----
if (-Not (Test-Path ".venv")) {
    Write-Host ""
    Write-Host "[1/4] Creating Python virtual environment..."
    py -m venv .venv
}
else {
    Write-Host ""
    Write-Host "[1/4] Python virtual environment already exists."
}

# 仮想環境有効化
& ".\venv\Scripts\Activate.ps1"

Write-Host ""
Write-Host "[2/4] Installing Python packages..."

python -m pip install --upgrade pip
pip install -r requirements.txt

#-----
# PlatformIO
#-----
Write-Host ""
Write-Host "[3/4] Checking PlatformIO..."

$PIO = $null

# PlatformIO Core
if (Test-Path "$env:USERPROFILE\.platformio\penv\Scripts\pio.exe") {
    $PIO = "$env:USERPROFILE\.platformio\penv\Scripts\pio.exe"
}
elseif (Get-Command pio -ErrorAction SilentlyContinue) {
    $PIO = (Get-Command pio).Source
}

if (-Not $PIO) {
    Write-Host ""
    Write-Host "ERROR: PlatformIO not found."
```

```

Write-Host "Please install the PlatformIO VSCode extension."
exit 1
}

Write-Host "Using: $PIO"

#-----
# PlatformIO Environment
#-----
Write-Host ""
Write-Host "[4/4] Checking PlatformIO environments..."

if (-Not (Test-Path ".pio\build\vehicle")) {
    Write-Host "Creating Vehicle environment..."
    & $PIO run -e vehicle
}
else {
    Write-Host "Vehicle environment already exists."
}

if (-Not (Test-Path ".pio\build\monitor")) {
    Write-Host "Creating Monitor environment..."
    & $PIO run -e monitor
}
else {
    Write-Host "Monitor environment already exists."
}

Write-Host ""
Write-Host "=====
Write-Host " Setup Complete!"
Write-Host "=====
Write-Host ""
Write-Host "To activate the Python environment:"
Write-Host ""
Write-Host ".\venv\Scripts\Activate.ps1"
Write-Host ""

```

リスト 4 setup.sh

```

#!/bin/bash

set -e

echo "=====
echo "    Project Setup"
echo "=====

#-----
# Python 仮想環境
#-----
if [ ! -d ".venv" ]; then

```

```

    echo ""
    echo "[1/4] Creating Python virtual environment..."
    python3 -m venv .venv
else
    echo ""
    echo "[1/4] Python virtual environment already exists."
fi

source .venv/bin/activate

echo ""
echo "[2/4] Installing Python packages..."

python -m pip install --upgrade pip
pip install -r requirements.txt

#-----
# PlatformIO
#-----
echo ""
echo "[3/4] Checking PlatformIO..."

if [ -x "$HOME/.platformio/penv/bin/pio" ]; then
    PIO="$HOME/.platformio/penv/bin/pio"
elif command -v pio >/dev/null 2>&1; then
    PIO=$(command -v pio)
else
    echo ""
    echo "ERROR: PlatformIO not found."
    echo "Please install the PlatformIO VSCode extension."
    exit 1
fi

echo "Using: $PIO"

#-----
# PlatformIO Environment
#-----
echo ""
echo "[4/4] Checking PlatformIO environments..."

if [ ! -d ".pio/build/vehicle" ]; then
    echo "Creating Vehicle environment..."
    "$PIO" run -e vehicle
else
    echo "Vehicle environment already exists."
fi

if [ ! -d ".pio/build/monitor" ]; then
    echo "Creating Monitor environment..."
    "$PIO" run -e monitor
else
    echo "Monitor environment already exists."

```

```

fi

echo ""
echo "=====
echo " Setup Complete!"
echo "=====
echo ""
echo "To activate the Python environment:"
echo ""
echo "source .venv/bin/activate"
echo ""

```

B. 衝突回避プロジェクトのヘッダファイル

衝突回避実験で用いる `include` ディレクトリ内のヘッダファイルを以下に示す.

表 2 衝突回避プロジェクトのヘッダファイル一覧

プログラム名	内容
<code>callbacks.hpp</code>	<code>espnow</code> の受信コールバック関数を定義するヘッダファイル
<code>computavoidance.hpp</code>	衝突回避速度算出関数を定義するヘッダファイル
<code>config.hpp</code>	コンフィグを定義するヘッダファイル
<code>controller.hpp</code>	コントローラを定義するヘッダファイル
<code>espnow.hpp</code>	ESP-NOW 関係の処理を定義するヘッダファイル
<code>logger.hpp</code>	記録処理を定義するヘッダファイル
<code>motor.hpp</code>	モータ制御を定義するヘッダファイル
<code>prediction.hpp</code>	交差点までの時間算出関数を定義するヘッダファイル
<code>sensor.hpp</code>	センサ処理を定義するヘッダファイル

以下に各ヘッダファイルをリスト 5, リスト 6, リスト 7, リスト 8, リスト 9, リスト 10, リスト 11, リスト 12, リスト 13 に示す.

リスト 5 `callbacks.hpp`

```

#ifndef CALLBACK_H
#define CALLBACK_H
#include "config.hpp"

void OnRecvData(const uint8_t *mac_addr, const uint8_t *data, int len);
#endif

```

リスト 6 `computavoidance.hpp`

```

#ifndef COMPUTAVOIDANCE_H
#define COMPUTAVOIDANCE_H
int computeAvoidanceSpeed(int OtherEnterTime, int OtherExitTime);

```

```
#endif
```

リスト 7 config.hpp

```
#ifndef CONFIG_H
#define CONFIG_H

#include <Arduino.h>
#include <LittleFS.h>
constexpr uint32_t INVALID_TIME = UINT32_MAX;

//-----
// ピン設定
//-----
constexpr int PHASE_PIN = D5;
constexpr int ENABLE_PIN = D3;
constexpr int SENSOR_PIN = D6;
constexpr int RUN_SWITCH_PIN = D2;

//-----
// PWM
//-----
constexpr int ledcChannel = 0;
constexpr int freq = 20000;
constexpr int resolution = 8;

//-----
// パラメータ
//-----
constexpr int INTERVAL_MS = 20;
constexpr int DEBOUNCE_US = 200;
constexpr int LINE_PITCH = 10;
constexpr int DIST_TO_INTERSECTION_ENTRY = 500; // 交差点入口までの距離 [mm]
constexpr int DIST_TO_INTERSECTION_EXIT = 700; // 交差点出口までの距離 [mm]
constexpr int VEHICLE_LENGTH = 150; // 車体全長 [mm]
constexpr int firstspeed = 100; // 初期速度[cm/s]

// メディアンフィルタパラメータ
constexpr int median_window_size = 7;

// PID コントローラパラメータ
constexpr float F_KP = 1.0;
constexpr float F_KI = 1.0;
constexpr float F_KD = 0.0;

constexpr int MAX_INTEGRAL = 200;

constexpr int MAX_PWM = 200;
constexpr int MIN_PWM = 10;
```

```

constexpr int MAX_SPEED = 400;
constexpr int MIN_SPEED = 20;

constexpr int margin_us=50000;

// 車両数
constexpr int vehiclecount = 2;

// 推定到着時間を発信するための閾値
constexpr int speedborder = 20;

//-----
// ログ構造体
//-----
struct StatusData{//espnow で共有するデータ
    uint32_t time_us;
    uint32_t speed;
    uint8_t pwm;
    uint32_t enterTime;
    uint32_t exitTime;
};

//-----
// extern
//-----

extern volatile int SensorlogIndex;
extern volatile int LineCount;
extern volatile uint8_t CurrentPWM;

extern volatile bool StartFlag;
extern volatile bool SaveFlag;
extern volatile bool SensorONFlag;

extern uint32_t START_US;
extern uint32_t LAST_US;

extern int CurrentSpeed;
extern int CullentPosition;
extern int targetSpeed;
extern int FirstTargetSpeed;

extern uint32_t enterTime;
extern uint32_t exitTime;

extern volatile bool RunFlag;
extern bool CollisionFlag;

extern float KP;
extern float KI;
extern float KD;

```

```
#endif // CONFIG_H
```

リスト 8 controller.hpp

```
#ifndef CONTROLLER_H
#define CONTROLLER_H

uint8_t PIDcontroller(int currentSpeed, int targetSpeed);
void ClearController();
#endif
```

リスト 9 espnow.hpp

```
#ifndef ESPNOW_HPP
#define ESPNOW_HPP

#include <Arduino.h>
#include <WiFi.h>
#include <esp_now.h>
#include "config.hpp"

// パケット種類
//=====
enum class PacketType : uint8_t{
    START = 0,
    FINISH,
    STATUS,
    SET_SPEED,
    GAIN
};

// パケット
//=====
struct PacketHead{
    PacketType type;
};

struct STARTPacket{
    PacketHead head;
    bool CollisionFlag;
};

struct FINISHPacket{
    PacketHead head;
};

struct STATUSPacket{
    PacketHead head;

    StatusData payload;
};
```

```

struct SETSPEEDPacket{
    PacketHead head;
    uint16_t targetSpeed;
};

struct GAINPacket{
    PacketHead head;
    float KP;
    float KI;
    float KD;
};

void setup_ESPNOW();

void SendSTART(bool collision);

void SendFINISH();

void SendSTATUS(const StatusData &data);

void SendSPEED(const int speed);

void SendGAIN(float kp,float ki,float kd);

#endif

```

リスト 10 logger.hpp

```

#ifndef LOGGER_HPP
#define LOGGER_HPP

#include"config.hpp"

void setup_logger();
void OpenFile();
void WriteLog(int i, StatusData data);
void CloseFile();
void sendLog();
#endif

```

リスト 11 motor.hpp

```

#ifndef MOTOR_H
#define MOTOR_H

void setup_motor();
void motorWrite(int duty);
void motorBrake();

#endif

```

リスト 12 prediction.hpp

```
#ifndef PREDICTION_H
#define PREDICTION_H
#include <Arduino.h>
uint32_t predictTimeToIntersection();
uint32_t predictTimeToExitIntersection();
#endif
```

リスト 13 sensor.hpp

```
#ifndef SENSOR_H
#define SENSOR_H
#include "config.hpp"

void setup_sensor();
void clearIntervalBuffer();
int getCurrentSpeed();
#endif
```

C. 衝突回避プロジェクトのソースディレクトリ

衝突回避実験で用いる **src** ディレクトリ内のディレクトリを以下に示す。PlatformIO では環境設定によってビルドするソースファイルをディレクトリ単位で設定できる。

表 3 衝突回避プロジェクトの **src**/ディレクトリ一覧

プログラム名	内容
common	env:monitor と env:vehicle で共通のディレクトリ
espnw	実験 1 で使用する espnow 通信確認用ディレクトリ
monitor	env:monitor 専用ディレクトリ
vehicle	env:vehicle 専用ディレクトリ

common ディレクトリ内のソースファイルを以下に示す。

表 4 衝突回避プロジェクトの **common** 内のソースファイル一覧

プログラム名	内容
config.cpp	基本的なパラメータをまとめたソースファイル
espnw.cpp	ESP-NOW に関する処理をまとめたソースファイル

common ディレクトリ内のソースファイルをリスト 14, リスト 15 に示す。

リスト 14 config.cpp

```
#include "config.hpp"

volatile int SensorlogIndex = 0;
```

```

volatile int LineCount = 0;

volatile uint8_t CurrentPWM = 0;

volatile bool StartFlag = false;
volatile bool SaveFlag = false;
volatile bool SensorONFlag = false;

uint32_t START_US = 0;
uint32_t LAST_US = 0;

int CurrentSpeed = 0;
int CurrentPosition = 0;
int targetSpeed = FirstTargetSpeed;
uint32_t enterTime;
uint32_t exitTime;

volatile bool RunFlag = false;
bool CollisionFlag = false;

int FirstTargetSpeed = 100;

float KP=F_KP;
float KI=F_KI;
float KD=F_KD;

```

リスト 15 espnow.cpp

```

#include "espnow.hpp"
#include "logger.hpp"
#include "motor.hpp"
#include "sensor.hpp"
#include "callbacks.hpp"

const uint8_t broadcastAddress[6] = {0xFF,0xFF,0xFF,0xFF,0xFF,0xFF};

//-----
// ESPNOW セットアップ関数
//-----
void setup_ESPNOW(){
    WiFi.mode(WIFI_STA);

    esp_now_init();

    esp_now_peer_info_t broadcast = {};
    memcpy(broadcast.peer_addr, broadcastAddress, 6);
    broadcast.channel = 0;
    broadcast.encrypt = false;

    esp_now_add_peer(&broadcast);

    esp_now_register_recv_cb(OnRecvData);
}

```

```

//-----
// START 送信関数
//-----
void SendSTART(bool collision){
    STARTPacket packet;

    packet.head.type = PacketType::START;
    packet.CollisionFlag = collision;

    esp_now_send(broadcastAddress,reinterpret_cast<uint8_t*>(&packet),sizeof(pack
et));
}

//-----
// FINISH 送信関数
//-----
void SendFINISH(){
    FINISHPacket packet;

    packet.head.type = PacketType::FINISH;

    esp_now_send(broadcastAddress,reinterpret_cast<uint8_t*>(&packet),sizeof(pack
et));
}

//-----
// STATUS 送信関数
//-----
void SendSTATUS(const StatusData &data){
    STATUSPacket packet;

    packet.head.type = PacketType::STATUS;
    packet.payload = data;

    esp_now_send(broadcastAddress,reinterpret_cast<uint8_t*>(&packet),sizeof(pack
et));
}

//-----
// SETSPEED 送信関数
//-----
void SendSPEED(const int speed){
    SETSPEEDPacket packet;
    packet.head.type=PacketType::SET_SPEED;
    packet.targetSpeed = speed;
    esp_now_send(broadcastAddress,reinterpret_cast<uint8_t*>(&packet),sizeof(pack
et));
}

//-----
// GAIN 送信関数

```

```
//-----
void SendGAIN(float kp,float ki,float kd){
    GAINPacket packet;
    packet.head.type=PacketType::GAIN;
    packet.KP=kp;
    packet.KI=ki;
    packet.KD=kd;
    esp_now_send(broadcastAddress,reinterpret_cast<uint8_t*>(&packet),sizeof(pack
et));
}
```

espnow ディレクトリ内のソースファイルを以下に示す.

表 5 衝突回避プロジェクトの espnow 内のソースファイル一覧

プログラム名	内容
main.cpp	ESP-NOW 通信実験用の main ファイル

common ディレクトリ内のソースファイルをリスト 16 に示す.

リスト 16 main.cpp

```
#include <Arduino.h>
#include <WiFi.h>
#include <esp_now.h>

// 実験 1 で使用する
// esp-now の導通確認実験

void AddPeer(const uint8_t *mac_addr);
void PrintMacAddress(const uint8_t *mac_addr);
void SendHello();
void SendAck(const uint8_t *mac_addr);
void SendPing(const uint8_t *mac_addr);
void SendPong(const uint8_t *mac_addr);
void SendDataRequest(const uint8_t *mac_addr);
void SendData(const uint8_t *mac_addr);
void OnDataRecv(const uint8_t *mac_addr, const uint8_t *data, int len);

uint32_t PINGSTART;
uint32_t DATASTART;

// パケット種別 (全パケット共通のヘッダとして使う)
enum PacketType : uint8_t{
    HELLO,
    ACK,
    PING,
    PONG,
    DATA_REQ, // データ要求
}
```

```

    DATA    // データ本体 (ダミーデータ)
};

// パケットヘッダ
// すべてのパケットは先頭にこのヘッダを持つ
struct PacketHeader{
    PacketType type;
};

// ヘッダのみのパケット (HELLO, ACK, PING, PONG, DATA_REQ 用)
struct Packet{
    PacketHeader header;
};

struct StatusData{//espnw で共有するデータ
    uint32_t time_us;
    uint32_t speed;
    uint8_t pwm;
    uint32_t enterTime;
    uint32_t exitTime;
};

// データパケット (ヘッダ + ダミーデータ本体)
struct DataPacket{
    PacketHeader header;
    StatusData payload;
};

uint8_t broadcastAddress[6] = {
    0xFF, 0xFF, 0xFF,
    0xFF, 0xFF, 0xFF
};

uint8_t unicastAddress[6];

void setup(){
    // シリアル通信開始
    Serial.begin(115200);

    // Wi-Fi を STA モードに設定
    WiFi.mode(WIFI_STA);

    // ESP-NOW 初期化
    if (esp_now_init() != ESP_OK) {
        Serial.println("ESP-NOW Init Failed");
        while (true);
    }

    // コールバック関数登録
    esp_now_register_rcv_cb(OnDataRecv);

    // ブロードキャストアドレスを Peer 登録

```

```

    AddPeer(broadcastAddress);

    Serial.println("ESP-NOW Ready");
}

//-----
// Peer 登録
//-----
void AddPeer(const uint8_t *mac_addr){
    Serial.print("Add Peer : ");
    PrintMacAddress(mac_addr);

    if(esp_now_is_peer_exist(mac_addr)) {
        Serial.println("Peer already exists");
        return;
    }

    esp_now_peer_info_t peerInfo = {};

    memcpy(peerInfo.peer_addr, mac_addr, 6);

    peerInfo.channel = 0;
    peerInfo.encrypt = false;

    esp_err_t result = esp_now_add_peer(&peerInfo);

    if(result == ESP_OK) {
        Serial.println("Peer Added");
    }
    else {
        Serial.print("Peer Add Failed : ");
        Serial.println(result);
    }
}

//-----
// HELLO 送信 (ブロードキャスト)
//-----
void SendHello(){
    Packet packet;
    packet.header.type = HELLO;

    Serial.println();
    if (esp_now_send(broadcastAddress, (uint8_t *)&packet, sizeof(packet)) ==
ESP_OK){
        Serial.println("Send : HELLO");
    }
    else{
        Serial.println("HELLO Send Failed");
    }
    Serial.println();
}

```

```

//-----
// MAC アドレス表示
//-----
void PrintMacAddress(const uint8_t *mac_addr){
    for (int i = 0; i < 6; i++){
        Serial.printf("%02X", mac_addr[i]);
        if (i != 5) Serial.print(":");
    }
    Serial.println();
}

//-----
// PING 送信 (ユニキャスト)
//-----
void SendPing(const uint8_t *mac_addr){
    Packet packet;
    packet.header.type = PING;
    PINGSTART = millis();

    Serial.println();
    Serial.print("Send : PING -> ");
    PrintMacAddress(mac_addr);

    if (esp_now_send(mac_addr, (uint8_t *)&packet, sizeof(packet)) == ESP_OK){
        Serial.println("PING Send OK");
    }
    else{
        Serial.println("PING Send Failed");
    }
    Serial.println();
}

//-----
// PONG 送信 (ユニキャスト)
//-----
void SendPong(const uint8_t *mac_addr){
    Packet packet;
    packet.header.type = PONG;

    Serial.print("Send : PONG -> ");
    PrintMacAddress(mac_addr);

    if (esp_now_send(mac_addr, (uint8_t *)&packet, sizeof(packet)) == ESP_OK){
        Serial.println("PONG Send OK");
    }
    else{
        Serial.println("PONG Send Failed");
    }
}

//-----
// DATA_REQ 送信 (ユニキャスト)

```

```

// 相手に対してデータ本体の送信を要求する
//-----
void SendDataRequest(const uint8_t *mac_addr){
    Packet packet;
    packet.header.type = DATA_REQ;
    DATASTART = millis();

    Serial.println();
    Serial.print("Send : DATA_REQ -> ");
    PrintMacAddress(mac_addr);

    if (esp_now_send(mac_addr, (uint8_t *)&packet, sizeof(packet)) == ESP_OK){
        Serial.println("DATA_REQ Send OK");
    }
    else{
        Serial.println("DATA_REQ Send Failed");
    }
    Serial.println();
}

//-----
// DATA 送信 (ユニキャスト)
// ダミーデータ本体を送信する
//-----
void SendData(const uint8_t *mac_addr){

    DataPacket packet;

    packet.header.type = DATA;

    // -----
    // ダミーデータ生成
    // -----

    static uint32_t counter = 0;

    packet.payload.time_us = micros();

    // 速度 500～1000 の範囲で変化
    packet.payload.speed = 500 + (counter % 500);

    // PWM 50～150
    packet.payload.pwm = 50 + (counter % 100);

    // 交差点突入時間
    packet.payload.enterTime = millis() + 1000;

    // 交差点退出時間
    packet.payload.exitTime = packet.payload.enterTime + 500;

    counter++;
}

```

```

Serial.println();
Serial.print("Send : DATA -> ");
PrintMacAddress(mac_addr);

Serial.println("Dummy Data:");
Serial.print(" time_us  : ");
Serial.println(packet.payload.time_us);

Serial.print(" speed   : ");
Serial.println(packet.payload.speed);

Serial.print(" pwm     : ");
Serial.println(packet.payload.pwm);

Serial.print(" enterTime : ");
Serial.println(packet.payload.enterTime);

Serial.print(" exitTime  : ");
Serial.println(packet.payload.exitTime);

if (esp_now_send(mac_addr,(uint8_t *)&packet,sizeof(packet)) == ESP_OK){
    Serial.println("DATA Send OK");
}
else{
    Serial.println("DATA Send Failed");
}
}

//-----
// 受信コールバック
//-----
void OnDataRecv(const uint8_t *mac_addr,const uint8_t *data,int len){
    // ヘッダ長すら無い場合は不正パケットとして破棄
    if (len < (int)sizeof(PacketHeader)){
        Serial.println("Invalid Packet Size");
        return;
    }

    // まずヘッダだけを見て種別を判定する
    const PacketHeader *header = (const PacketHeader *)data;

    switch(header->type){
        case HELLO:{
            if (len != sizeof(Packet)){
                Serial.println("Invalid HELLO Packet Size");
                break;
            }

            Serial.println();
            Serial.print("Recv : HELLO from ");

            PrintMacAddress(mac_addr);

```

```

        Serial.println();
        break;
    }

    case PING:{
        if (len != sizeof(Packet)){
            Serial.println("Invalid PING Packet Size");
            break;
        }

        Serial.println();
        Serial.print("Recv : PING from ");
        PrintMacAddress(mac_addr);

        // PONG 返信
        SendPong(mac_addr);
        Serial.println();
        break;
    }

    case PONG:{
        if (len != sizeof(Packet)){
            Serial.println("Invalid PONG Packet Size");
            break;
        }

        uint32_t elapsed = millis() - PINGSTART;

        Serial.println();
        Serial.print("Recv : PONG from ");
        PrintMacAddress(mac_addr);

        Serial.print("RTT : ");
        Serial.print(elapsed);
        Serial.println(" ms");
        Serial.println();
        break;
    }

    case DATA_REQ:{
        if (len != sizeof(Packet)){
            Serial.println("Invalid DATA_REQ Packet Size");
            break;
        }

        Serial.println();
        Serial.print("Recv : DATA_REQ from ");
        PrintMacAddress(mac_addr);

        // 要求されたのでデータ本体を返信
        SendData(mac_addr);
    }

```

```

        Serial.println();
        break;
    }

    case DATA:{
        if (len != sizeof(DataPacket)){
            Serial.println("Invalid DATA Packet Size");
            break;
        }

        uint32_t elapsed = millis() - DATASTART;

        Serial.println();
        Serial.print("Recv : DATA from ");
        PrintMacAddress(mac_addr);

        //ダミーデータ表示 (data を DataPacket でキャスト)

        DataPacket *recvPacket = (DataPacket *)data;

        Serial.println("DATA:");

        Serial.print(" time_us  :");
        Serial.println(recvPacket->payload.time_us);

        Serial.print(" speed    :");
        Serial.println(recvPacket->payload.speed);

        Serial.print(" pwm      :");
        Serial.println(recvPacket->payload.pwm);

        Serial.print(" enterTime :");
        Serial.println(recvPacket->payload.enterTime);

        Serial.print(" exitTime  :");
        Serial.println(recvPacket->payload.exitTime);

        Serial.print("RTT : ");
        Serial.print(elapsed);
        Serial.println(" ms");
        Serial.println();
        break;
    }

    default:{
        Serial.println();
        Serial.println("Unknown Packet");
        Serial.println();
        break;
    }
}
}
}

```

```

//-----
// メインループ
//-----
void loop(){
    if (Serial.available()){
        String command = Serial.readStringUntil('\n');
        command.trim();

        // HELLO 送信
        if (command == "HELLO"){
            SendHello();
        }

        // Peer 登録
        else if (command.startsWith("addpeer")){
            uint8_t mac[6];

            // "addpeer "以降を取得
            String macString = command.substring(8);

            if (sscanf(macString.c_str(),
                "%hhx:%hhx:%hhx:%hhx:%hhx:%hhx",
                &mac[0], &mac[1], &mac[2],
                &mac[3], &mac[4], &mac[5]) == 6){
                Serial.print("Add Peer : ");
                PrintMacAddress(mac);

                AddPeer(mac);
            }
            else{
                Serial.println("Invalid MAC Address");
            }
        }

        // PING 送信
        else if (command.startsWith("PING")){
            uint8_t mac[6];

            // "PING "以降を取得
            String macString = command.substring(5);

            if (sscanf(macString.c_str(),
                "%hhx:%hhx:%hhx:%hhx:%hhx:%hhx",
                &mac[0], &mac[1], &mac[2],
                &mac[3], &mac[4], &mac[5]) == 6){
                SendPing(mac);
            }
            else{
                Serial.println("Invalid MAC Address");
            }
        }
    }
}

```

```

// DATAREQ 送信
else if (command.startsWith("DATAREQ")){
    uint8_t mac[6];

    // "DATAREQ "以降を取得
    String macString = command.substring(8);

    if (sscanf(macString.c_str(),
               "%hhx:%hhx:%hhx:%hhx:%hhx:%hhx",
               &mac[0], &mac[1], &mac[2],
               &mac[3], &mac[4], &mac[5]) == 6){
        SendDataRequest(mac);
    }
    else{
        Serial.println("Invalid MAC Address");
    }
}

else{
    Serial.println("Unknown Command");
}
}
}

```

monitor ディレクトリ内のソースファイルを以下に示す.

表 6 衝突回避プロジェクトの monitor 内のソースファイル一覧

プログラム名	内容
main.cpp	env:monitor 用の main ファイル
callbacks.cpp	env:monitor 用の受信コールバック関数のソースファイル
logger.cpp	ログ保存処理のソースファイル

monitor ディレクトリ内のソースファイルをリスト 17, リスト 18, リスト 19 に示す.

リスト 17 main.cpp

```

#include <Arduino.h>
#include "config.hpp"
#include "espnow.hpp"
#include "logger.hpp"

// 実験 2,実験 3 で使用するモニタ用
//

void setup(){
    Serial.begin(115200);
    setup_ESPNOW();
    setup_logger();
}

```

```

// メインループ
void loop(){
    if (!Serial.available()) return;

    // コマンド入力
    String cmd = Serial.readStringUntil('\n');
    cmd.trim();

    if (cmd == "start"){
        SendSTART(false);
        OpenFile();
        Serial.println("Send START (CollisionFlag = false)");
    }
    else if (cmd == "start collision"){
        SendSTART(true);
        Serial.println("Send START (CollisionFlag = true)");
    }
    else if (cmd.startsWith("speed ")){
        // "speed "以降を整数に変換
        int speed = cmd.substring(6).toInt();
        SendSPEED(speed);
        Serial.printf("Send SET_SPEED (%d)\n", speed);
    }
    else if (cmd.startsWith("gain ")){
        float kp, ki, kd;

        // "gain KP KI KD" を解析
        int ret = sscanf(cmd.c_str(), "gain %f %f %f", &kp, &ki, &kd);

        if (ret == 3){
            SendGAIN(kp, ki, kd);
            Serial.printf("Send GAIN (KP=%.3f, KI=%.3f, KD=%.3f)\n", kp, ki, kd);
        }
        else{
            Serial.println("Usage: gain <KP> <KI> <KD>");
        }
    }
    else if (cmd == "finish"){
        SendFINISH();
        CloseFile();
        sendLog();
        Serial.println("Send FINISH");
    }
    else{
        Serial.println("Unknown command");
    }
}

```

リスト 18 callbacks.cpp

```
#include "config.hpp"
```

```

#include "espnow.hpp"
#include "logger.hpp"
#include "callbacks.hpp"
//モニター側
uint8_t vehicleaddress[vehiclecount][6];
bool addressRegistered[vehiclecount] = {false};
//-----
// START パケット受信処理関数
//-----
void OnStartPacket(bool collision){
    return;
}

//-----
// FINISH パケット受信処理関数
//-----
void OnFinishPacket(){
    return;
}

//-----
// 車両番号取得（未登録なら登録）
//-----
int GetVehicleIndex(const uint8_t *mac_addr){

    // 既登録か検索
    for(int i = 0; i < vehiclecount; i++){

        if(addressRegistered[i] &&
            memcmp(vehicleaddress[i], mac_addr, 6) == 0){

            return i;
        }
    }

    // 未登録なら空を探す
    for(int i = 0; i < vehiclecount; i++){

        if(!addressRegistered[i]){

            memcpy(vehicleaddress[i], mac_addr, 6);
            addressRegistered[i] = true;

            Serial.printf("Vehicle %d registered\n", i);

            return i;
        }
    }

    // 登録できない
    return -1;
}

```

```

void OnStatusPacket(const uint8_t *mac_addr, StatusData payload)
{
    int index = GetVehicleIndex(mac_addr);

    if(index < 0){
        Serial.println("Vehicle table full.");
        return;
    }

    Serial.println("-----");
    Serial.printf("Vehicle   : %d\n", index);
    Serial.printf("MAC Address : %02X:%02X:%02X:%02X:%02X:%02X\n",
        mac_addr[0], mac_addr[1], mac_addr[2],
        mac_addr[3], mac_addr[4], mac_addr[5]);

    Serial.printf("Time   : %lu\n", payload.time_us);
    Serial.printf("Speed  : %d\n", payload.speed);
    Serial.printf("PWM    : %d\n", payload.pwm);
    Serial.println("-----");

    WriteLog(index, payload);
}

//-----
// SETSPEED パケット受信処理関数
//-----
void OnSetSpeedPacket(int speed){
    return;
}

//-----
// GAIN パケット受信処理関数
//-----
void OnGAINPacket(float kp,float ki,float kd){
    return ;
}

//-----
// モニター受信コールバック関数
//-----
void OnRecvData(const uint8_t *mac_addr,const uint8_t *data,int len){
    const PacketHead *head=reinterpret_cast<const PacketHead *>(data);

    if (head->type != PacketType::STATUS) return;

    STATUSPacket packet;
    memcpy(&packet, data, sizeof(packet));
    StatusData payload=packet.payload;

    OnStatusPacket(mac_addr,payload);
}

```

リスト 19 logger.cpp

```
#include <Arduino.h>
#include <LittleFS.h>
#include "config.hpp"
#include "logger.hpp"

//モニタ用
File file[vehiclecount];
bool fileHasData[vehiclecount]; // 実際に STATUS を受信して記録したかどうか

//-----
//フラッシュメモリクリア関数
//-----
void clearFlash(){
    if (!LittleFS.begin()){
        return ;
    }

    File root = LittleFS.open("/");
    File file = root.openNextFile();

    while (file){
        String path = "/" + String(file.name());
        file.close();

        LittleFS.remove(path);

        file = root.openNextFile();
    }
}

//-----
// セットアップ関数
//-----
void setup_logger(){
    if (!LittleFS.begin(true)) {
        Serial.println("LittleFS mount failed");
    }
    clearFlash();
}

//-----
// ファイル展開関数 (START)
//-----
void OpenFile(){
    for (int i = 0; i < vehiclecount; i++) {

        fileHasData[i] = false; // 新しい試行のためリセット
    }
}
```

```

String filename = "/log" + String(i) + ".csv";

file[i] = LittleFS.open(filename, FILE_WRITE);

if (!file[i]) {
    Serial.printf("Failed to open %s\n", filename.c_str());
    continue;
}

Serial.printf("%s opened\n", filename.c_str());

// ヘッダ
file[i].println("Time_s,Speed,PWM");
}
}

//-----
// ログ書き込み関数 (STATUS 受信)
//-----
void WriteLog(int i, StatusData data){

    if (!file[i]) return;

    fileHasData[i] = true; // このファイルは実データを持つ

    file[i].print(data.time_us);
    file[i].print(",");
    file[i].print(data.speed);
    file[i].print(",");
    file[i].println(data.pwm);
}

//-----
// ファイル閉じ関数 (FINISH)
//-----
void CloseFile(){

    for(int i = 0; i < vehiclecount; i++){
        if(file[i]){
            file[i].close();
        }
    }

    Serial.println("All logs ready.");
}

//-----
// ログ転送関数
//-----
void sendLog(){

    // 実際にデータがある車両数を数える

```

```

int sendCount = 0;
for(int i = 0; i < vehiclecount; i++){
    if (fileHasData[i]) sendCount++;
}

Serial.printf("COUNT %d\n", sendCount);

for(int i = 0; i < vehiclecount; i++){

    if (!fileHasData[i]) continue; // 記録がない車両はスキップ

    String filename = "/log" + String(i) + ".csv";

    File f = LittleFS.open(filename, FILE_READ);

    if(!f){
        Serial.printf("Cannot open %s\n", filename.c_str());
        continue;
    }

    Serial.printf("BEGIN %d\n", i);

    while(f.available()){
        Serial.write(f.read());
    }

    Serial.println();
    Serial.printf("END %d\n", i);

    f.close();
}

// Python から ACK 待ち
uint32_t start = millis();

while(millis() - start < 5000){

    if(Serial.available()){

        String cmd = Serial.readStringUntil('\n');
        cmd.trim();

        if(cmd == "ACK"){

            for(int i = 0; i < vehiclecount; i++){
                if (!fileHasData[i]) continue; // 記録がある分だけ削除

                String filename = "/log" + String(i) + ".csv";
                LittleFS.remove(filename);
            }

            Serial.println("ALL LOGS DELETED");
        }
    }
}

```

```

        break;
    }
}
}
}

```

vehicle ディレクトリ内のソースファイルを以下に示す.

表 7 衝突回避プロジェクトの vehicle 内のソースファイル一覧

プログラム名	内容
main.cpp	env:vehicle 用の main ファイル
callbacks.cpp	env:vehicle 用の受信コールバック関数のソースファイル
computavoidance.cpp	衝突回避速度算出用ソースファイル
controller.cpp	コントローラ用ソースファイル
prediction.cpp	交差点までの時間を算出するソースファイル
motor.cpp	モータ制御を行うソースファイル
sesor.cpp	センサ処理を行うソースファイル

vehicle ディレクトリ内のソースファイルをリスト 20, リスト 21, リスト 22, リスト 23, リスト 24, リスト 25, リスト 26 に示す.

リスト 20 main.cpp

```

#include <Arduino.h>
#include "config.hpp"
#include "espnow.hpp"
#include "motor.hpp"
#include "sensor.hpp"
#include "controller.hpp"
#include "prediction.hpp"

// 実験 2,実験 3 で使用する車両用

StatusData CurrentSTATUS;

void setup(){
    Serial.begin(115200);
    setup_ESPNOW();
    setup_motor();
    setup_sensor();
}

// メインループ
void loop()
{
    if(RunFlag){
        CurrentSpeed=getCurrentSpeed();
        CurrentPWM = PIDcontroller(CurrentSpeed, targetSpeed);
    }
}

```

```

    motorWrite(CurrentPWM);
    enterTime=predictTimeToIntersection();
    exitTime=predictTimeToExitIntersection();
    CurrentSTATUS.time_us=micros() - START_US;
    CurrentSTATUS.speed = CurrentSpeed;
    CurrentSTATUS.pwm=CurrentPWM;
    CurrentSTATUS.enterTime=enterTime;
    CurrentSTATUS.exitTime=exitTime;
    SendSTATUS(CurrentSTATUS);
}
else{
    motorBrake();
}
delay(INTERVAL_MS);
}

```

リスト 21 callbacs.cpp

```

#include "espnow.hpp"
#include "motor.hpp"
#include "sensor.hpp"
#include "callbacks.hpp"
#include "controller.hpp"
#include "computavoidance.hpp"
//ビークル側

//-----
// START パケット受信処理関数
//-----
void OnStartPacket(bool collision){
    START_US=micros();
    RunFlag = true;
    CollisionFlag = collision;
    targetSpeed=FirstTargetSpeed;
    ClearController();
    clearIntervalBuffer();
}

//-----
// FINISH パケット受信処理関数
//-----
void OnFinishPacket(){
    RunFlag = false;
    return;
}

//-----
// STATUS パケット受信処理関数
//-----
void OnStatusPacket(const uint8_t *mac_addr,StatusData payload){

```

```

        if (payload.enterTime == INVALID_TIME || payload.exitTime ==
INVALID_TIME){
            return;
        }

        targetSpeed = computeAvoidanceSpeed(payload.enterTime,payload.exitTime);

        return;
    }

//-----
// SETSPEED パケット受信処理関数
//-----
void OnSetSpeedPacket(int speed){
    targetSpeed=speed;
    FirstTargetSpeed = speed;
    return;
}

//-----
// GAIN パケット受信処理関数
//-----
void OnGAINPacket(float kp,float ki,float kd){
    KP = kp;
    KI = ki;
    KD = kd;
    return ;
}

//-----
// ビークル受信コールバック関数
//-----
void OnRecvData(const uint8_t *mac_addr,const uint8_t *data,int len){
    const PacketHead *head=reinterpret_cast<const PacketHead *>(data);

    switch (head->type){
        case PacketType::START:{
            const STARTPacket *packet=reinterpret_cast<const STARTPacket *>(data);
            OnStartPacket(packet->CollisionFlag);
            break;
        }
        case PacketType::FINISH:{
            OnFinishPacket();
            break;
        }
        case PacketType::STATUS:{
            STATUSPacket packet;
            memcpy(&packet, data, sizeof(packet));
            StatusData payload=packet.payload;
            OnStatusPacket(mac_addr,payload);
            break;
        }
    }
}

```

```

    }

    case PacketType::SET_SPEED:{
        const SETSPEEDPacket *packet=reinterpret_cast<const SETSPEEDPacket
*>(data);
        int speed = packet->targetSpeed;
        OnSetSpeedPacket(speed);
        break;
    }
    case PacketType::GAIN:{
        const GAINPacket *packet=reinterpret_cast<const GAINPacket *>(data);
        float kp=packet->KP;
        float ki=packet->KI;
        float kd=packet->KD;
        OnGAINPacket(kp,ki,kd);
        break;
    }
}
}
}

```

リスト 22 computavoidamce.cpp

```

#include "espnow.hpp"
#include "motor.hpp"
#include "sensor.hpp"
#include "callbacks.hpp"
#include "controller.hpp"

//-----
// 衝突回避速度算出関数
//-----
int computeAvoidanceSpeed(int OtherEnterTime,int OtherExitTime){
    int MyEnterTime = enterTime;
    int MyExitTime = exitTime;
    int NewTargetSpeed ;
    uint32_t now = micros() - START_US;
    ////////////
    uint32_t desiredEnter;
    if (CollisionFlag){
        // 相手と同時に突入
        desiredEnter = OtherEnterTime;
    }
    else if (OtherEnterTime>MyEnterTime){
        //自分が相手より早く突入
        desiredEnter = enterTime - margin_us;    //50ms 早く 突入
    }
    else{
        //自分が相手より遅く突入
        desiredEnter = OtherExitTime + margin_us; // 50ms 遅く 突入
    }
}

```

```

    if (desiredEnter <= now) return targetSpeed;

    int remain = DIST_TO_INTERSECTION_ENTRY - LineCount * LINE_PITCH;

    if (remain <= 0) return targetSpeed;

    uint32_t remainTime = desiredEnter - now;

    NewTargetSpeed = constrain(remain * 1000000UL /
remainTime,MIN_SPEED,MAX_SPEED);
    ////////////
    return NewTargetSpeed;
}

```

リスト 23 controller.cpp

```

#include "config.hpp"

float integral=0;
float previousError=0;

// 課題としてコントローラを設計させる

//-----
// コントローラ関数
//-----
uint8_t PIDcontroller(int currentSpeed, int targetSpeed){

    // P 制御
    float error = targetSpeed - currentSpeed;
    float P = KP * error;

    // I 制御
    integral += error * INTERVAL_MS / 1000 ;
    integral = constrain(integral,-MAX_INTEGRAL,MAX_INTEGRAL);
    float I = KI * integral;

    // D 制御
    float derivative = (error - previousError) *1000/ INTERVAL_MS ;
    float D = KD * derivative;
    previousError = error;

    float pwm = P + I + D;
    pwm = constrain(pwm,MIN_PWM,MAX_PWM);

    return (uint8_t)pwm;
}

//-----
// コントローラ初期化関数
//-----
void ClearController(){

```

```

integral=0;
previousError=0;
targetSpeed = firstspeed;
}

```

リスト 24 prediction.cpp

```

#include <Arduino.h>
#include "config.hpp"

//-----
// 交差点突入時間計算関数
//-----
uint32_t predictTimeToIntersection() {

    int currentPosition = LineCount * LINE_PITCH; // [mm]
    int speed = CurrentSpeed; // [cm/s]

    // 停止中は計算不可
    if (speed <= 0 || abs(targetSpeed-CurrentSpeed)<speedborder){
        return UINT32_MAX;
    }

    // 交差点入口までの残り距離 [mm]
    int remainingDistance = DIST_TO_INTERSECTION_ENTRY - currentPosition;

    // すでに到達済み
    if (remainingDistance <= 0){
        return 0;
    }

    // [us]
    return (uint32_t)((uint64_t)remainingDistance * 1000000ULL / (speed * 10));
}

uint32_t predictTimeToExitIntersection(){

    int currentPosition = LineCount * LINE_PITCH; // [mm]
    int speed = CurrentSpeed; // [cm/s]

    // 停止中は計算不可
    if (speed <= 0 || abs(targetSpeed-CurrentSpeed)<speedborder){
        return UINT32_MAX;
    }

    // 車体後端が交差点を抜けるまでの残り距離 [mm]
    int remainingDistance =
        DIST_TO_INTERSECTION_EXIT + VEHICLE_LENGTH - currentPosition;

    // 既に通過済み
    if (remainingDistance <= 0){
        return 0;
    }
}

```

```

    }

    // 時間[us]
    return (uint32_t)((uint64_t)remainingDistance * 1000000ULL / (speed * 10));
}

```

リスト 25 motor.cpp

```

#include <Arduino.h>
#include "config.hpp"
#include "motor.hpp"

//-----
// モータセットアップ関数
//-----
void setup_motor(){
    //ピンをセット
    pinMode(PHASE_PIN, OUTPUT);
    pinMode(SENSOR_PIN, INPUT);
    if(RUN_SWITCH_PIN) pinMode(RUN_SWITCH_PIN, OUTPUT);

    if(RUN_SWITCH_PIN) digitalWrite(RUN_SWITCH_PIN, HIGH);

    // 回転方向
    digitalWrite(PHASE_PIN, LOW);

    // PWM をセット
    ledcSetup(ledcChannel, freq, resolution);
    ledcAttachPin(ENABLE_PIN, ledcChannel);

    motorBrake();
}

//-----
// モータ速度関数
//-----
void motorWrite(int duty){
    ledcWrite(ledcChannel, duty);
}

//-----
// モータブレーキ関数
//-----
void motorBrake(){
    // モータドライバにショートブレーキを指示する
    ledcWrite(ledcChannel, 0);
}

```

リスト 26 sensor.cpp

```

#include <Arduino.h>
#include "config.hpp"

```

```

#include "sensor.hpp"

uint32_t intervalBuffer[median_window_size];

int head = 0;
int count = 0;

//-----
// センサログ追加関数
//-----
void pushInterval(uint32_t interval){
    intervalBuffer[head] = interval;

    head = (head + 1) % median_window_size;

    if (count < median_window_size){
        count++;
    }
}

//-----
// センサログクリア関数
//-----
void clearIntervalBuffer(){
    head = 0;
    count = 0;
}

//-----
// メディアンフィルタ関数
//-----
uint32_t medianInterval(){
    if (count == 0){
        return 100000000;
    }

    uint32_t work[median_window_size];

    for (int i = 0; i < count; i++){
        work[i] = intervalBuffer[i];
    }

    for (int i = 0; i < count - 1; i++){
        for (int j = i + 1; j < count; j++){
            if (work[i] > work[j]){
                uint32_t tmp = work[i];
                work[i] = work[j];
                work[j] = tmp;
            }
        }
    }
}

```

```

    return work[count / 2];
}

//-----
// 現在速度取得関数
//-----
int getCurrentSpeed(){
    int speed;
    uint32_t median_interval = medianInterval();
    uint32_t now_interval = micros() - LAST_US;
    uint32_t true_interval = max(median_interval, now_interval);
    speed = LINE_PITCH * 100000 / true_interval;
    return speed;
}

//-----
// ライン検出関数
//-----
void IRAM_ATTR sensorISR(){
    bool state = digitalRead(SENSOR_PIN);

    //入力が HIGH でセンサーON フラグが false のとき記録をとる
    if(state && !SensorONFlag){
        //現在時間を取得
        uint32_t NOW_US = micros();
        uint32_t Interval_US = NOW_US - LAST_US;

        //最新時間からデバウンス時間経過していないなら無視
        if(Interval_US > DEBOUNCE_US){
            //ライン検出をインクリメント
            LineCount++;
            //ログデータを配列に格納
            pushInterval(Interval_US);

            //最新時間を更新
            LAST_US = NOW_US;
            //インデックスを更新
            SensorlogIndex++;
            SensorONFlag = true;
        }
    }

    //入力が LOW ならセンサーON フラグを false にする
    else if(!state){
        SensorONFlag = false;
    }
}

//-----
// センサセットアップ
//-----
void setup_sensor(){
    //ピンをセット
    pinMode(SENSOR_PIN, INPUT);
}

```

```
// ライン検出関数の割り込みをセット
attachInterrupt(digitalPinToInterrupt(SENSOR_PIN),sensorISR,CHANGE);
}
```

C. 衝突回避プロジェクトのシリアルモニタプログラム

衝突回避実験で用いるシリアルモニタのプログラムを以下に示す.

表 8 シリアルモニタのプログラム一覧

プログラム名	内容
exp1_monitor.py	実験 1 で使用するシリアルモニタのプログラム
exp2_monitor.py	実験 2 で使用するシリアルモニタのプログラム
exp3_monitor.py	実験 3 で使用するシリアルモニタのプログラム

以下にシリアルモニタのプログラムをリスト 27, リスト 28, リスト 29 に示す.

リスト 27 exp1_monitor.py

```
"""
exp1_monitor.py

実験 1 (ESP-NOW 導通確認実験) 用のシリアルモニタ GUI

必要ライブラリ
  pip install pyserial
"""
import queue
import serial
import serial.tools.list_ports

import threading

import tkinter as tk
from tkinter import ttk
from tkinter.scrolledtext import ScrolledText

# =====
# SerialManager
# =====
class SerialManager:

    def __init__(self):

        self.ser = None

        self.running = False
```

```

self.receive_thread = None

# GUI へ渡すメッセージ
self.message_queue = queue.Queue()

# -----
# GUI へログ表示
# -----
def log(self, text):

    print(text)

    self.message_queue.put(text)

# -----
# GUI 用メッセージ取得
# -----
def get_message(self):

    try:
        return self.message_queue.get_nowait()
    except queue.Empty:
        return None

# -----
# COM ポート一覧取得
# -----
@staticmethod
def get_ports():

    ports = serial.tools.list_ports.comports()

    return [port.device for port in ports]

# -----
# 接続
# -----
def connect(self, port, baudrate=115200):

    if self.ser is not None:
        return True

    try:

        self.ser = serial.Serial(
            port=port,
            baudrate=baudrate,
            timeout=0.1
        )

        self.running = True

```

```

        self.receive_thread = threading.Thread(
            target=self.receive_loop,
            daemon=True
        )

        self.receive_thread.start()

        self.log(f"Connected : {port}")

        return True

    except Exception as e:

        self.log(f"Connection Error : {e}")

        return False

# -----
# 切断
# -----
def disconnect(self):

    self.running = False

    if self.ser is not None:

        try:
            self.ser.close()
        except:
            pass

        self.ser = None

    self.log("Disconnected")

# -----
# コマンド送信
# -----
def send(self, command):

    if self.ser is None:
        return

    try:

        self.ser.write((command + "\n").encode())

        self.log("> " + command)

    except Exception as e:

        self.log(f"Send Error : {e}")

```

```

# -----
# HELLO 送信 (ブロードキャスト)
# -----
def send_hello(self):

    self.send("HELLO")

# -----
# Peer 登録
# -----
def send_addpeer(self, mac):

    self.send(f"addpeer {mac}")

# -----
# PING 送信
# -----
def send_ping(self, mac):

    self.send(f"PING {mac}")

# -----
# DATA_REQ 送信
# -----
def send_datareq(self, mac):

    self.send(f"DATAREQ {mac}")

# -----
# 受信スレッド
# -----
def receive_loop(self):

    while self.running:

        if self.ser is None:
            break

        try:

            line = self.ser.readline()

            if len(line) == 0:
                continue

            text = line.decode(
                errors="ignore"
            ).strip()

            if text == "":
                continue

```

```

        self.log(text)

    except Exception as e:

        self.log(f"Receive Error : {e}")

        self.disconnect()

        break

# =====
# MonitorGUI
# =====
class MonitorGUI:

    #=====
    # 初期化
    #=====
    def __init__(self):

        self.serial = SerialManager()

        self.root = tk.Tk()
        self.root.title("ESP-NOW Monitor (実験 1)")
        self.root.geometry("700x550")

        #-----
        # COM ポート
        #-----
        top = ttk.Frame(self.root)
        top.pack(fill="x", padx=10, pady=10)

        ttk.Label(
            top,
            text="COM Port"
        ).pack(side="left")

        self.port_var = tk.StringVar()

        self.port_box = ttk.Combobox(
            top,
            width=20,
            textvariable=self.port_var,
            state="readonly"
        )

        self.port_box.pack(side="left", padx=5)

        ttk.Button(
            top,
            text="Refresh",

```

```

        command=self.refresh_ports
    ).pack(side="left")

    self.connect_button = ttk.Button(
        top,
        text="Connect",
        command=self.connect
    )

    self.connect_button.pack(side="left", padx=10)

    #-----
    # HELLO (ブロードキャスト)
    #-----
    hello_frame = ttk.Frame(self.root)
    hello_frame.pack(fill="x", padx=10, pady=5)

    ttk.Button(
        hello_frame,
        text="HELLO (Broadcast)",
        command=self.send_hello,
        width=25
    ).pack(side="left")

    #-----
    # MAC アドレス指定 + 各種送信
    #-----
    mac_frame = ttk.Frame(self.root)
    mac_frame.pack(fill="x", padx=10, pady=10)

    ttk.Label(
        mac_frame,
        text="MAC Address"
    ).pack(side="left")

    self.mac_var = tk.StringVar(value="XX:XX:XX:XX:XX:XX")

    self.mac_entry = ttk.Entry(
        mac_frame,
        width=20,
        textvariable=self.mac_var
    )

    self.mac_entry.pack(side="left", padx=5)

    button_frame = ttk.Frame(self.root)
    button_frame.pack(fill="x", padx=10)

    ttk.Button(
        button_frame,
        text="Add Peer",
        command=self.send_addpeer,
        width=15

```

```

).pack(side="left", padx=5, pady=5)

ttk.Button(
    button_frame,
    text="PING",
    command=self.send_ping,
    width=15
).pack(side="left", padx=5, pady=5)

ttk.Button(
    button_frame,
    text="DATA REQUEST",
    command=self.send_datareq,
    width=15
).pack(side="left", padx=5, pady=5)

#-----
# 任意コマンド送信
#-----
raw_frame = ttk.Frame(self.root)
raw_frame.pack(fill="x", padx=10, pady=10)

ttk.Label(
    raw_frame,
    text="Command"
).pack(side="left")

self.raw_var = tk.StringVar()

self.raw_entry = ttk.Entry(
    raw_frame,
    width=40,
    textvariable=self.raw_var
)

self.raw_entry.pack(side="left", padx=5)
self.raw_entry.bind("<Return>", lambda e: self.send_raw())

ttk.Button(
    raw_frame,
    text="Send",
    command=self.send_raw
).pack(side="left", padx=5)

#-----
# ログ表示
#-----
ttk.Label(
    self.root,
    text="Log"
).pack(anchor="w", padx=10)

self.log = ScrolledText(

```

```

        self.root,
        height=20,
        state="disabled"
    )

    self.log.pack(
        fill="both",
        expand=True,
        padx=10,
        pady=10
    )

    # COM 一覧取得
    self.refresh_ports()

    # Queue 監視開始
    self.root.after(
        50,
        self.update_log
    )

    # ×ボタン
    self.root.protocol(
        "WM_DELETE_WINDOW",
        self.close
    )

#=====
# COM 一覧更新
#=====
def refresh_ports(self):

    ports = self.serial.get_ports()

    self.port_box["values"] = ports

    if len(ports) > 0:
        self.port_box.current(0)

#=====
# 接続
#=====
def connect(self):

    if self.serial.ser is None:

        port = self.port_var.get()

        if port == "":
            return

        if self.serial.connect(port):

```

```

        self.connect_button.config(
            text="Disconnect",
            command=self.disconnect
        )

    else:

        self.disconnect()

#=====
# 切断
#=====
def disconnect(self):

    self.serial.disconnect()

    self.connect_button.config(
        text="Connect",
        command=self.connect
    )

#=====
# HELLO
#=====
def send_hello(self):

    self.serial.send_hello()

#=====
# Add Peer
#=====
def send_addpeer(self):

    mac = self.mac_var.get().strip()

    if mac == "":
        self.append_log("Invalid MAC Address")
        return

    self.serial.send_addpeer(mac)

#=====
# PING
#=====
def send_ping(self):

    mac = self.mac_var.get().strip()

    if mac == "":
        self.append_log("Invalid MAC Address")
        return

```

```

        self.serial.send_ping(mac)

#=====
# DATA REQUEST
#=====
def send_datareq(self):

    mac = self.mac_var.get().strip()

    if mac == "":
        self.append_log("Invalid MAC Address")
        return

    self.serial.send_datareq(mac)

#=====
# 任意コマンド送信
#=====
def send_raw(self):

    command = self.raw_var.get().strip()

    if command == "":
        return

    self.serial.send(command)

    self.raw_var.set("")

#=====
# ログ更新
#=====
def update_log(self):

    while True:

        message = self.serial.get_message()

        if message is None:
            break

        self.append_log(message)

    # 50ms 後に再度チェック
    self.root.after(50, self.update_log)

#=====
# ログ追加
#=====
def append_log(self, text):

    self.log.configure(state="normal")

```

```

        self.log.insert(tk.END, text + "\n")

        self.log.see(tk.END)

        self.log.configure(state="disabled")

#=====
# 終了処理
#=====
def close(self):

    try:
        self.serial.disconnect()
    except:
        pass

    self.root.destroy()

#=====
# GUI 開始
#=====
def run(self):

    self.root.mainloop()

#=====
# main
#=====
def main():
    app = MonitorGUI()
    app.run()

if __name__ == "__main__":
    main()

```

リスト 28 exp2_monitor.py

```

"""
esp32_monitor.py

必要ライブラリ
    pip install pyserial
"""
import queue
import serial
import os
import serial.tools.list_ports

import threading
from datetime import datetime

```

```

import tkinter as tk
from tkinter import ttk
from tkinter.scrolledtext import ScrolledText

# =====
# SerialManager
# =====
class SerialManager:

    def __init__(self):

        self.ser = None

        self.running = False

        self.receive_thread = None

        # GUI へ渡すメッセージ
        self.message_queue = queue.Queue()

        # CSV 受信
        self.receiving_csv = False
        self.current_vehicle = None
        self.csv_buffer = []

        # 転送予定ファイル数の管理
        self.expected_count = None
        self.received_count = 0

        # -----
        # GUI へログ表示
        # -----
        def log(self, text):

            print(text)

            self.message_queue.put(text)

        # -----
        # GUI 用メッセージ取得
        # -----
        def get_message(self):

            try:
                return self.message_queue.get_nowait()
            except queue.Empty:
                return None

        # -----
        # COM ポート一覧取得
        # -----
        @staticmethod

```

```

def get_ports():

    ports = serial.tools.list_ports.comports()

    return [port.device for port in ports]

# -----
# 接続
# -----
def connect(self, port, baudrate=115200):

    if self.ser is not None:
        return True

    try:

        self.ser = serial.Serial(
            port=port,
            baudrate=baudrate,
            timeout=0.1
        )

        self.running = True

        self.receive_thread = threading.Thread(
            target=self.receive_loop,
            daemon=True
        )

        self.receive_thread.start()

        self.log(f"Connected : {port}")

        return True

    except Exception as e:

        self.log(f"Connection Error : {e}")

        return False

# -----
# 切断
# -----
def disconnect(self):

    self.running = False

    if self.ser is not None:

        try:
            self.ser.close()
        except:

```

```

        pass

        self.ser = None

        self.log("Disconnected")

# -----
# コマンド送信
# -----
def send(self, command):

    if self.ser is None:
        return

    try:

        self.ser.write((command + "\n").encode())

        self.log("> " + command)

    except Exception as e:

        self.log(f"Send Error : {e}")

# -----
# START
# -----
def send_start(self):

    self.send("start")

# -----
# FINISH
# -----
def send_finish(self):

    self.send("finish")

# -----
# SET SPEED
# -----
def send_speed(self, speed):

    self.send(f"speed {speed}")

# -----
# SET GAIN
# -----
def send_gain(self, kp, ki, kd):
    self.send(f"gain {kp} {ki} {kd}")

# -----
# 受信スレッド

```

```

# -----
def receive_loop(self):

    while self.running:

        if self.ser is None:
            break

        try:

            line = self.ser.readline()

            if len(line) == 0:
                continue

            text = line.decode(
                errors="ignore"
            ).strip()

            if text == "":
                continue

            # -----
            # 転送予定件数
            # -----
            if text.startswith("COUNT"):

                parts = text.split()

                if len(parts) == 2:
                    self.expected_count = int(parts[1])
                else:
                    self.expected_count = None

                self.received_count = 0

                self.log(f'Expecting {self.expected_count} log file(s)')

                continue

            # -----
            # CSV 開始
            # -----
            if text.startswith("BEGIN"):

                parts = text.split()

                if len(parts) == 2:
                    self.current_vehicle = int(parts[1])
                else:
                    self.current_vehicle = 0

                self.receiving_csv = True

```

```

        self.csv_buffer.clear()

        self.log(f'Receiving Vehicle {self.current_vehicle}')

        continue

# -----
# CSV 終了
# -----
if text.startswith("END"):

    self.save_csv(self.current_vehicle)

    self.receiving_csv = False
    self.current_vehicle = None

    self.received_count += 1

    # 予定件数すべて受信したら ACK を返す
    if (self.expected_count is not None
        and self.received_count >= self.expected_count):

        self.send("ACK")
        self.expected_count = None
        self.received_count = 0

    continue

# -----
# 全転送終了
# -----
if text == "ALL LOGS DELETED":
    self.log(text)
    continue

# -----
# CSV 受信
# -----
if self.receiving_csv:
    self.csv_buffer.append(text)
else:
    self.log(text)

except Exception as e:

    self.log(f'Receive Error : {e}')

    self.disconnect()

    break

# -----

```

```

# CSV 保存
# -----
def save_csv(self, vehicle):

    os.makedirs("log", exist_ok=True)

    timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")

    filename = os.path.join(
        "log",
        f"{timestamp}_vehicle{vehicle}.csv"
    )

    try:

        with open(filename, "w", encoding="utf-8") as f:

            for line in self.csv_buffer:
                f.write(line + "\n")

            self.log(f"Saved : {filename}")

    except Exception as e:

        self.log(f"Save Error : {e}")

# =====
# MonitorGUI
# =====
class MonitorGUI:

    # =====
    # 初期化
    # =====
    def __init__(self):

        self.serial = SerialManager()

        self.root = tk.Tk()
        self.root.title("ESP32 Monitor")
        self.root.geometry("700x550")

        #-----
        # COM ポート
        #-----
        top = ttk.Frame(self.root)
        top.pack(fill="x", padx=10, pady=10)

        ttk.Label(
            top,
            text="COM Port"
        ).pack(side="left")

```

```

self.port_var = tk.StringVar()

self.port_box = ttk.Combobox(
    top,
    width=20,
    textvariable=self.port_var,
    state="readonly"
)

self.port_box.pack(side="left", padx=5)

ttk.Button(
    top,
    text="Refresh",
    command=self.refresh_ports
).pack(side="left")

self.connect_button = ttk.Button(
    top,
    text="Connect",
    command=self.connect
)

self.connect_button.pack(side="left", padx=10)

#-----
# 操作ボタン
#-----
button_frame = ttk.Frame(self.root)
button_frame.pack(fill="x", padx=10)

self.start_button = ttk.Button(
    button_frame,
    text="START",
    command=self.send_start,
    width=25
)

self.start_button.pack(pady=5)

self.finish_button = ttk.Button(
    button_frame,
    text="FINISH",
    command=self.send_finish,
    width=25
)

self.finish_button.pack(pady=5)

#-----
# 目標速度設定
#-----

```

```

speed_frame = ttk.Frame(self.root)
speed_frame.pack(fill="x", padx=10, pady=10)

ttk.Label(
    speed_frame,
    text="Target Speed"
).pack(side="left")

self.speed_var = tk.StringVar(value="100")

self.speed_entry = ttk.Entry(
    speed_frame,
    width=10,
    textvariable=self.speed_var
)

self.speed_entry.pack(side="left", padx=5)

ttk.Button(
    speed_frame,
    text="SET SPEED",
    command=self.send_speed
).pack(side="left", padx=5)

# -----
# PID ゲイン設定
# -----
gain_frame = ttk.Frame(self.root)
gain_frame.pack(fill="x", padx=10, pady=5)

ttk.Label(
    gain_frame,
    text="KP"
).pack(side="left")

self.kp_var = tk.StringVar(value="1.0")
ttk.Entry(
    gain_frame,
    width=8,
    textvariable=self.kp_var
).pack(side="left", padx=3)

ttk.Label(
    gain_frame,
    text="KI"
).pack(side="left")

self.ki_var = tk.StringVar(value="1.0")
ttk.Entry(
    gain_frame,
    width=8,
    textvariable=self.ki_var
).pack(side="left", padx=3)

```

```

ttk.Label(
    gain_frame,
    text="KD"
).pack(side="left")

self.kd_var = tk.StringVar(value="0.0")
ttk.Entry(
    gain_frame,
    width=8,
    textvariable=self.kd_var
).pack(side="left", padx=3)

ttk.Button(
    gain_frame,
    text="SET GAIN",
    command=self.send_gain
).pack(side="left", padx=10)

#-----
# ログ表示
#-----
ttk.Label(
    self.root,
    text="Log"
).pack(anchor="w", padx=10)

self.log = ScrolledText(
    self.root,
    height=20,
    state="disabled"
)

self.log.pack(
    fill="both",
    expand=True,
    padx=10,
    pady=10
)

# COM 一覧取得
self.refresh_ports()

# Queue 監視開始
self.root.after(
    50,
    self.update_log
)

# ×ボタン
self.root.protocol(
    "WM_DELETE_WINDOW",

```

```

        self.close
    )

#=====
# COM 一覧更新
#=====
def refresh_ports(self):

    ports = self.serial.get_ports()

    self.port_box["values"] = ports

    if len(ports) > 0:
        self.port_box.current(0)

#=====
# 接続
#=====
def connect(self):

    if self.serial.ser is None:

        port = self.port_var.get()

        if port == "":
            return

        if self.serial.connect(port):

            self.connect_button.config(
                text="Disconnect",
                command=self.disconnect
            )

        else:

            self.disconnect()

#=====
# 切断
#=====
def disconnect(self):

    self.serial.disconnect()

    self.connect_button.config(
        text="Connect",
        command=self.connect
    )

#=====
# START

```

```

#=====
def send_start(self):

    self.serial.send_start()

#=====
# FINISH
#=====
def send_finish(self):

    self.serial.send_finish()

#=====
# SET SPEED
#=====
def send_speed(self):

    try:
        speed = int(self.speed_var.get())
    except ValueError:
        self.append_log("Invalid speed")
        return

    self.serial.send_speed(speed)

#=====
# SET GAIN
#=====
def send_gain(self):

    try:
        kp = float(self.kp_var.get())
        ki = float(self.ki_var.get())
        kd = float(self.kd_var.get())
    except ValueError:
        self.append_log("Invalid gain")
        return

    self.serial.send_gain(kp, ki, kd)

#=====
# ログ更新
#=====
def update_log(self):

    while True:

        message = self.serial.get_message()

        if message is None:
            break

        self.append_log(message)

```

```

        # 50ms 後に再度チェック
        self.root.after(50, self.update_log)

#=====
# ログ追加
#=====
def append_log(self, text):

    self.log.configure(state="normal")

    self.log.insert(tk.END, text + "\n")

    self.log.see(tk.END)

    self.log.configure(state="disabled")

#=====
# 終了処理
#=====
def close(self):

    try:
        self.serial.disconnect()
    except:
        pass

    self.root.destroy()

#=====
# GUI 開始
#=====
def run(self):

    self.root.mainloop()

#=====
# main
#=====
def main():
    app = MonitorGUI()
    app.run()

if __name__ == "__main__":
    main()

```

リスト 29 exp3_monitor.py

```

"""
exp3_monitor.py

実験 3 用シリアルモニタ

```

- START / FINISH
- 衝突フラグ（トグルスイッチ）
 - ON : START 押下時に "start collision" を送信
 - OFF : START 押下時に "start" を送信

必要ライブラリ

```
pip install pyserial
```

```
"""
```

```
import queue
```

```
import serial
```

```
import os
```

```
import serial.tools.list_ports
```

```
import threading
```

```
from datetime import datetime
```

```
import tkinter as tk
```

```
from tkinter import ttk
```

```
from tkinter.scrolledtext import ScrolledText
```

```
# =====
```

```
# SerialManager
```

```
# =====
```

```
class SerialManager:
```

```
    def __init__(self):
```

```
        self.ser = None
```

```
        self.running = False
```

```
        self.receive_thread = None
```

```
        # GUI へ渡すメッセージ
```

```
        self.message_queue = queue.Queue()
```

```
        # CSV 受信
```

```
        self.receiving_csv = False
```

```
        self.current_vehicle = None
```

```
        self.csv_buffer = []
```

```
        # 転送予定ファイル数の管理
```

```
        self.expected_count = None
```

```
        self.received_count = 0
```

```
# -----
```

```
# GUI へログ表示
```

```
# -----
```

```
def log(self, text):
```

```
    print(text)
```

```

        self.message_queue.put(text)

# -----
# GUI 用メッセージ取得
# -----
def get_message(self):

    try:
        return self.message_queue.get_nowait()
    except queue.Empty:
        return None

# -----
# COM ポート一覧取得
# -----
@staticmethod
def get_ports():

    ports = serial.tools.list_ports.comports()

    return [port.device for port in ports]

# -----
# 接続
# -----
def connect(self, port, baudrate=115200):

    if self.ser is not None:
        return True

    try:

        self.ser = serial.Serial(
            port=port,
            baudrate=baudrate,
            timeout=0.1
        )

        self.running = True

        self.receive_thread = threading.Thread(
            target=self.receive_loop,
            daemon=True
        )

        self.receive_thread.start()

        self.log(f"Connected : {port}")

        return True

    except Exception as e:

```

```

        self.log(f"Connection Error : {e}")

        return False

# -----
# 切断
# -----
def disconnect(self):

    self.running = False

    if self.ser is not None:

        try:
            self.ser.close()
        except:
            pass

        self.ser = None

    self.log("Disconnected")

# -----
# コマンド送信
# -----
def send(self, command):

    if self.ser is None:
        return

    try:

        self.ser.write((command + "\n").encode())

        self.log("> " + command)

    except Exception as e:

        self.log(f"Send Error : {e}")

# -----
# START (衝突フラグに応じて分岐)
# -----
def send_start(self, collision):

    if collision:
        self.send("start collision")
    else:
        self.send("start")

# -----

```

```

# FINISH
# -----
def send_finish(self):

    self.send("finish")

# -----
# 受信スレッド
# -----
def receive_loop(self):

    while self.running:

        if self.ser is None:
            break

        try:

            line = self.ser.readline()

            if len(line) == 0:
                continue

            text = line.decode(
                errors="ignore"
            ).strip()

            if text == "":
                continue

            # -----
            # 転送予定件数
            # -----
            if text.startswith("COUNT"):

                parts = text.split()

                if len(parts) == 2:
                    self.expected_count = int(parts[1])
                else:
                    self.expected_count = None

                self.received_count = 0

                self.log(f"Expecting {self.expected_count} log file(s)")

                continue

            # -----
            # CSV 開始
            # -----
            if text.startswith("BEGIN"):

```

```

parts = text.split()

if len(parts) == 2:
    self.current_vehicle = int(parts[1])
else:
    self.current_vehicle = 0

self.receiving_csv = True
self.csv_buffer.clear()

self.log(f'Receiving Vehicle {self.current_vehicle}')

continue

# -----
# CSV 終了
# -----
if text.startswith("END"):

    self.save_csv(self.current_vehicle)

    self.receiving_csv = False
    self.current_vehicle = None

    self.received_count += 1

    # 予定件数すべて受信したら ACK を返す
    if (self.expected_count is not None
        and self.received_count >= self.expected_count):

        self.send("ACK")
        self.expected_count = None
        self.received_count = 0

    continue

# -----
# 全転送終了
# -----
if text == "ALL LOGS DELETED":
    self.log(text)
    continue

# -----
# CSV 受信中
# -----
if self.receiving_csv:
    self.csv_buffer.append(text)
else:
    self.log(text)

```

```

        except Exception as e:

            self.log(f"Receive Error : {e}")

            self.disconnect()

            break

# -----
# CSV 保存
# -----
def save_csv(self, vehicle):

    os.makedirs("log", exist_ok=True)

    timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")

    filename = os.path.join(
        "log",
        f"{timestamp}_vehicle{vehicle}.csv"
    )

    try:

        with open(filename, "w", encoding="utf-8") as f:

            for line in self.csv_buffer:
                f.write(line + "\n")

            self.log(f"Saved : {filename}")

    except Exception as e:

        self.log(f"Save Error : {e}")

# =====
# MonitorGUI
# =====
class MonitorGUI:

    #=====
    # 初期化
    #=====
    def __init__(self):

        self.serial = SerialManager()

        self.root = tk.Tk()
        self.root.title("ESP32 Monitor - Experiment 3")
        self.root.geometry("700x550")

    #-----

```

```

# COM ポート
#-----
top = ttk.Frame(self.root)
top.pack(fill="x", padx=10, pady=10)

ttk.Label(
    top,
    text="COM Port"
).pack(side="left")

self.port_var = tk.StringVar()

self.port_box = ttk.Combobox(
    top,
    width=20,
    textvariable=self.port_var,
    state="readonly"
)

self.port_box.pack(side="left", padx=5)

ttk.Button(
    top,
    text="Refresh",
    command=self.refresh_ports
).pack(side="left")

self.connect_button = ttk.Button(
    top,
    text="Connect",
    command=self.connect
)

self.connect_button.pack(side="left", padx=10)

#-----
# 衝突フラグ (トグルスイッチ)
#-----
collision_frame = ttk.Frame(self.root)
collision_frame.pack(fill="x", padx=10, pady=5)

ttk.Label(
    collision_frame,
    text="Collision Flag"
).pack(side="left")

self.collision_var = tk.BooleanVar(value=False)

self.collision_switch = ttk.Checkbutton(
    collision_frame,
    text="OFF",
    variable=self.collision_var,
    command=self.toggle_collision

```

```

)

self.collision_switch.pack(side="left", padx=10)

#-----
# 操作ボタン
#-----
button_frame = ttk.Frame(self.root)
button_frame.pack(fill="x", padx=10, pady=10)

self.start_button = ttk.Button(
    button_frame,
    text="START",
    command=self.send_start,
    width=25
)

self.start_button.pack(pady=5)

self.finish_button = ttk.Button(
    button_frame,
    text="FINISH",
    command=self.send_finish,
    width=25
)

self.finish_button.pack(pady=5)

#-----
# ログ表示
#-----
ttk.Label(
    self.root,
    text="Log"
).pack(anchor="w", padx=10)

self.log = ScrolledText(
    self.root,
    height=20,
    state="disabled"
)

self.log.pack(
    fill="both",
    expand=True,
    padx=10,
    pady=10
)

# COM 一覧取得
self.refresh_ports()

```

```

# Queue 監視開始
self.root.after(
    50,
    self.update_log
)

# ×ボタン
self.root.protocol(
    "WM_DELETE_WINDOW",
    self.close
)

#=====
# COM 一覧更新
#=====
def refresh_ports(self):

    ports = self.serial.get_ports()

    self.port_box["values"] = ports

    if len(ports) > 0:
        self.port_box.current(0)

#=====
# 接続
#=====
def connect(self):

    if self.serial.ser is None:

        port = self.port_var.get()

        if port == "":
            return

        if self.serial.connect(port):

            self.connect_button.config(
                text="Disconnect",
                command=self.disconnect
            )

        else:

            self.disconnect()

#=====
# 切断
#=====
def disconnect(self):

```

```

self.serial.disconnect()

self.connect_button.config(
    text="Connect",
    command=self.connect
)

#=====
# 衝突フラグ切替表示
#=====
def toggle_collision(self):

    if self.collision_var.get():
        self.collision_switch.config(text="ON")
    else:
        self.collision_switch.config(text="OFF")

#=====
# START (衝突フラグの状態を反映)
#=====
def send_start(self):

    self.serial.send_start(self.collision_var.get())

#=====
# FINISH
#=====
def send_finish(self):

    self.serial.send_finish()

#=====
# ログ更新
#=====
def update_log(self):

    while True:

        message = self.serial.get_message()

        if message is None:
            break

        self.append_log(message)

    # 50ms 後に再度チェック
    self.root.after(50, self.update_log)

#=====
# ログ追加
#=====
def append_log(self, text):

```

```

self.log.configure(state="normal")

self.log.insert(tk.END, text + "\n")

self.log.see(tk.END)

self.log.configure(state="disabled")

#=====
# 終了処理
#=====
def close(self):

    try:
        self.serial.disconnect()
    except:
        pass

    self.root.destroy()

#=====
# GUI 開始
#=====
def run(self):

    self.root.mainloop()

# =====
# main
# =====
def main():
    app = MonitorGUI()
    app.run()

if __name__ == "__main__":
    main()

```